

GYMRIDE

User Flows & UAT Test Cases

Web partner and admin panels

Customer mobile application

Backend financial and authorization boundaries

AS-BUILT SCOPE

Phases 1-6 | 65 acceptance scenarios | Local development guide

Contents

Contents	2
1. Product scope and actors	3
Explicitly excluded	3
2. Local UAT environment	3
Development identities	3
3. End-to-end marketplace flow	4
3.1 Customer mobile	4
3.2 Partner web	4
3.3 Admin web	4
4. Critical booking and payment states	5
5. UAT execution rules	5
6. Shared authentication and session cases	6
7. Partner web cases	8
8. Admin web cases	11
9. Customer mobile cases	14
10. Cross-surface integrity cases	18
11. UAT execution record	20
12. Acceptance exit criteria	21

1. Product scope and actors

This guide reflects the implemented repository through Phase 6. The customer mobile app supports discovery, reservations, payments and account management. The web surfaces are operational consoles for gym partners and administrators. There is no customer-facing web app.

Surface	Primary actor	Purpose
Customer mobile app	Customer	Discover gyms, select a branch and plan, reserve capacity, pay, and manage bookings/profile
Partner web panel	Gym owner or authorized manager	Configure gyms, branches, plans, schedules, capacity, bookings, earnings, and settlements
Admin web panel	Admin or super admin	Review gyms, inspect bookings, operate development payments/refunds/settlements, and reconcile finance
Backend API	All surfaces and payment/payout providers	Enforce identity, authorization, state transitions, capacity, commercial snapshots, and financial integrity

Explicitly excluded

QR check-in, reviews, push notifications, hybrid plans, recommendations, AI features and production deployment are future-phase work and are not included in these flows or acceptance claims.

2. Local UAT environment

Component	Local address
Backend and Swagger	http://localhost:3000/api/v1 and http://localhost:3000/api/docs
Admin panel	http://localhost:3001
Partner panel	http://localhost:3002
Mobile app	Expo/React Native development build or Expo Go, configured for the local API

Development identities

Role	Phone	Authentication
Admin	+919876543210	Request OTP; use dynamically displayed development OTP
Gym owner / partner	+919876543211	Request OTP; use dynamically displayed development OTP
Customer	+919876543212	Request OTP; use dynamically displayed development OTP

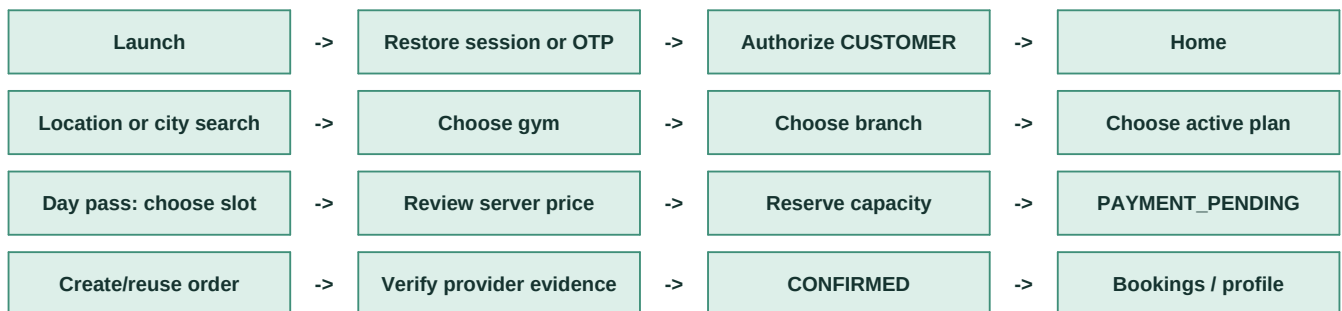
OTPs are generated per request, expire after five minutes and are single-use. Development payment and payout providers simulate outcomes only; they do not move external money.

3. End-to-end marketplace flow

The marketplace becomes customer-ready only after the partner configuration and admin approval paths are complete.

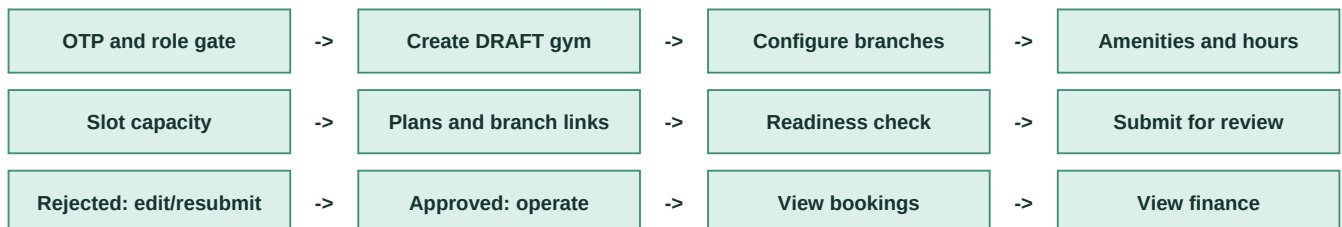
PARTNER WEB	OTP login -> Draft gym -> Branches, amenities and hours -> Slots and plans -> Submit
ADMIN WEB	Review owner and gym readiness -> Approve or reject -> Audit decision
CUSTOMER MOBILE	OTP login -> Discover -> Gym and branch -> Plan and slot -> Reserve -> Pay -> Booking
BACKEND FINANCE	Verify provider evidence -> Confirm atomically -> Earning and ledger -> Refund / settlement -> Reconcile

3.1 Customer mobile

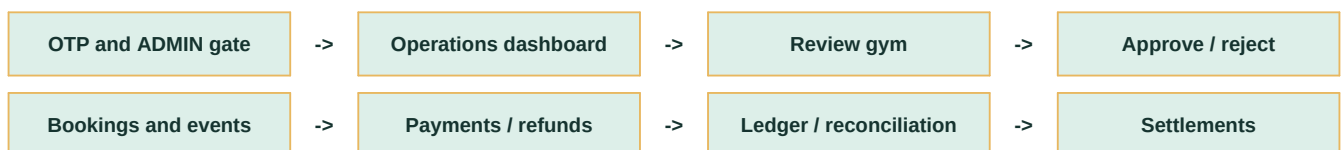


Denied location permission falls back to city search. A sold-out or blocked slot cannot be selected. Cancellation or expiry releases capacity. A customer-controlled success flag can never confirm a booking.

3.2 Partner web



3.3 Admin web



4. Critical booking and payment states

From	Trigger	To
PAYMENT_PENDING	Verified payment SUCCESS	CONFIRMED
PAYMENT_PENDING	Terminal payment failure	PAYMENT_FAILED
PAYMENT_PENDING	Customer cancellation	CANCELLED
PAYMENT_PENDING	Reservation deadline	EXPIRED
CONFIRMED / CANCELLED / EXPIRED	Completed full refund	REFUNDED

Stage	Valid progression
Order	CREATED -> PENDING -> AUTHORIZED -> SUCCESS
Failure	PENDING -> FAILED / CANCELLED / EXPIRED
Refund	SUCCESS -> REFUND_PENDING -> PARTIALLY_REFUNDED or REFUNDED

A partial refund changes the payment, earning and ledger values but must not mark the whole booking REFUNDED. Critical financial updates are transactional, idempotent and protected by database constraints.

5. UAT execution rules

- Execute P0 before release and P1 before phase acceptance. Schedule P2 unless it covers a changed area.
- Record build, device/browser, tester, result and evidence for every case.
- Use unique gym names and idempotency keys. Never delete financial history to reset a test.
- Use two independent partner-owned gyms for IDOR tests.
- Claim Razorpay only after official test credentials, signed callbacks and a native mobile build were validated.

6. Shared authentication and session cases

AUTH-01 | P0 | Admin OTP login

PRECONDITIONS	Development backend; seeded admin
STEPS	1. Open Admin 2. enter admin phone 3. request OTP 4. enter displayed OTP
EXPECTED	Dashboard opens; profile contains ADMIN/SUPER_ADMIN; token values are not rendered

AUTH-02 | P0 | Partner OTP login

PRECONDITIONS	Development backend; seeded partner
STEPS	1. Open Partner 2. request and verify OTP
EXPECTED	Partner dashboard opens and only authorized gym data is visible

AUTH-03 | P0 | Customer mobile OTP login

PRECONDITIONS	Mobile app points to local API
STEPS	1. Enter customer phone 2. request OTP 3. enter displayed code
EXPECTED	Home opens; user has CUSTOMER; refresh credential is stored through the platform storage adapter

AUTH-04 | P0 | Invalid/expired OTP

PRECONDITIONS	An OTP was requested
STEPS	1. Enter an incorrect code 2. then test an expired code
EXPECTED	Safe INVALID_OTP/OTP_EXPIRED message; no session; attempts/rate limits remain enforced

AUTH-05 | P0 | Wrong-role portal access

PRECONDITIONS	Valid customer or partner OTP
STEPS	1. Try customer in Admin 2. try customer in Partner 3. try admin/partner-only identity in mobile
EXPECTED	Access is denied and local session is cleared; protected data is never displayed

AUTH-06 | P1 | Session restoration

PRECONDITIONS	Successfully signed in
STEPS	1. Reload web tab 2. restart mobile process
EXPECTED	Same authorized session restores through refresh; profile is loaded before protected navigation

AUTH-07 | P0 | Logout during API failure

PRECONDITIONS	Authenticated; simulate offline/logout failure
STEPS	Select Log out
EXPECTED	Local tokens/query data are cleared and login screen appears even if server logout fails

AUTH-08 | P1 | Concurrent refresh

PRECONDITIONS	Authenticated session with expired access token
STEPS	Trigger multiple API reads at once
EXPECTED	One refresh operation is shared; requests recover without token-rotation race

7. Partner web cases

PWEB-01 | P1 | Dashboard totals

PRECONDITIONS	Partner logged in
STEPS	Open Dashboard
EXPECTED	Total gyms/branches and status counts match owned resources only; loading/error states are usable

PWEB-02 | P0 | Create a gym

PRECONDITIONS	Partner logged in
STEPS	1. Open Create gym 2. submit empty/invalid form 3. then enter valid name/description
EXPECTED	Validation blocks invalid data; valid gym is created as DRAFT and detail page opens

PWEB-03 | P0 | Create and edit a branch

PRECONDITIONS	Editable gym exists
STEPS	1. Add address, city, state, postal code, coordinates and timezone 2. save 3. edit
EXPECTED	Valid data persists; invalid coordinates/email/phone are rejected; duplicate branch name is rejected

PWEB-04 | P1 | Configure amenities

PRECONDITIONS	Branch exists
STEPS	1. Select amenities 2. save 3. reload
EXPECTED	Selected amenities persist and only known amenity IDs are accepted

PWEB-05 | P0 | Configure operating hours

PRECONDITIONS	Branch exists
STEPS	1. Add normal hours 2. try overlapping and close-before-open periods 3. save valid schedule
EXPECTED	Invalid periods are blocked; valid branch-local hours persist

PWEB-06 | P0 | Configure slot inventory

PRECONDITIONS	Active branch has hours
STEPS	1. Set slot duration, capacity, booking window and minimum advance 2. inspect date
EXPECTED	Future slots follow local hours; capacity/available totals are correct; invalid values fail

PWEB-07 | P0 | Create and activate plan

PRECONDITIONS	Gym has branch
STEPS	1. Create day pass with rupee price and branch 2. activate it
EXPECTED	Price is persisted exactly in paise; active plan becomes customer-visible for assigned branch

PWEB-08 | P0 | Block incomplete review submission

PRECONDITIONS	Draft gym lacks a required branch/hour
STEPS	Submit for review
EXPECTED	Submission fails with stable missing-requirement labels; gym remains editable DRAFT

PWEB-09 | P0 | Successful submission and review lock

PRECONDITIONS	All requirements complete
STEPS	1. Confirm Submit for review 2. attempt edit
EXPECTED	Gym becomes PENDING_APPROVAL; review-state profile is read-only

PWEB-10 | P1 | Rejection correction and resubmission

PRECONDITIONS	Admin rejected gym with reason
STEPS	1. Open gym 2. read feedback 3. edit 4. resubmit
EXPECTED	Reason is visible; rejected gym is editable; new submission becomes PENDING_APPROVAL

PWEB-11 | P0 | Authorized booking filters

PRECONDITIONS	Approved gym has bookings
STEPS	1. Filter by gym, branch, status and date 2. paginate 3. open detail
EXPECTED	Only authorized bookings appear; filters persist in URL; lifecycle and commercial snapshot are correct

PWEB-12 | P0 | Partner finance views

PRECONDITIONS	Authorized gym has successful payment
STEPS	1. Select gym 2. inspect Overview, Payments, Earnings, Settlements 3. filter/paginate/open details
EXPECTED	Aggregates match records; no provider secrets; earnings cannot be modified

PWEB-13 | P0 | Partner finance IDOR

PRECONDITIONS	Partner A and Partner B each own a gym
STEPS	As A, request B's booking/payment/earning/settlement IDs and B's gym filter
EXPECTED	Every request is denied or not found; no B metadata or totals leak

PWEB-14 | P1 | Branch status control

PRECONDITIONS	Partner owns branch
STEPS	Deactivate then reactivate branch
EXPECTED	Status updates; suspended branches remain non-editable by partner; discovery respects active status

8. Admin web cases

AWEB-01 | P1 | Operations dashboard

PRECONDITIONS	Admin logged in
STEPS	Open Dashboard and pending review list
EXPECTED	Counts match platform data; pending link and loading/error states work

AWEB-02 | P0 | Approve complete gym

PRECONDITIONS	Gym is PENDING_APPROVAL
STEPS	1. Review owner, branches, hours and plans 2. approve 3. confirm
EXPECTED	Gym becomes APPROVED; action appears once in audit; gym becomes discoverable

AWEB-03 | P0 | Reject gym with reason

PRECONDITIONS	Gym is PENDING_APPROVAL
STEPS	1. Choose Reject 2. submit missing/short reason 3. then valid reason
EXPECTED	Invalid reason blocked; gym becomes REJECTED; reason/audit visible to partner

AWEB-04 | P0 | Suspend and reactivate

PRECONDITIONS	Gym is APPROVED
STEPS	1. Suspend with reason 2. verify discovery 3. reactivate
EXPECTED	Suspended gym is removed from discovery; both audited transitions follow state rules

AWEB-05 | P1 | Platform booking inspection

PRECONDITIONS	Bookings in several states
STEPS	1. Filter by status/gym/branch/dates 2. paginate 3. open detail
EXPECTED	Read-only results and lifecycle events match database; no customer contact details leak

AWEB-06 | P0 | Development payment capture

PRECONDITIONS	Development payment is PENDING
STEPS	1. Finance -> Payments 2. open payment 3. confirm simulation
EXPECTED	Payment becomes SUCCESS; booking becomes CONFIRMED; earning and required ledger entries appear once; no real money moves

AWEB-07 | P0 | Partial refund

PRECONDITIONS	Successful captured payment exists
STEPS	1. Request a valid partial refund in paise 2. confirm
EXPECTED	Refund succeeds/pends per provider; refunded total and earning deduction update; booking is not REFUNDED

AWEB-08 | P0 | Full/excess/duplicate refund

PRECONDITIONS	Partially or fully refundable payment exists
STEPS	1. Request remaining full amount 2. then excess and duplicate requests
EXPECTED	Full refund yields payment/booking refund semantics; excess returns REFUND_AMOUNT_EXCEEDED; duplicates do not double-post

AWEB-09 | P0 | Settlement generation and processing

PRECONDITIONS	Eligible unsettled earnings exist
STEPS	1. Generate period settlement 2. retry same request 3. process READY settlement
EXPECTED	Eligible earnings included once; retry is idempotent; development payout is clearly simulated; status reaches PAID

AWEB-10 | P0 | Settlement reversal

PRECONDITIONS	Settlement is PAID
STEPS	1. Enter reason 2. confirm reversal 3. retry
EXPECTED	Audited reversal entry is appended once; payable liability restored; original ledger/history unchanged

AWEB-11 | P0 | Ledger immutability

PRECONDITIONS	Financial activity exists
STEPS	1. Inspect ledger 2. attempt unsupported edit/delete via UI/API
EXPECTED	Entries are read-only/append-only; correction requires a compensating entry

AWEB-12 | P0 | Reconciliation report

PRECONDITIONS	Create or fixture an inconsistent state in isolated test DB
STEPS	Run/inspect reconciliation
EXPECTED	Finding code and entity ID are reported; scan does not silently mutate finance data

AWEB-13 | P1 | Finance filters and pagination

PRECONDITIONS	More than one page of finance records
STEPS	Apply gym/branch/date/status filters and navigate pages
EXPECTED	Totals/list results honor filters; settlement ignores branch filter by design; empty/error states are clear

9. Customer mobile cases

MOB-01 | P0 | First-time OTP login

PRECONDITIONS	Fresh install/session
STEPS	1. Enter +919876543212 2. request code 3. verify displayed development code
EXPECTED	Home appears and a customer account/session is created safely

MOB-02 | P1 | OTP resend and input rules

PRECONDITIONS	On verification screen
STEPS	1. Enter letters/too few digits 2. try resend during 45-second cooldown 3. resend afterward
EXPECTED	Only six digits accepted; verify disabled until valid; cooldown prevents rapid resend

MOB-03 | P0 | Restore and revoke session

PRECONDITIONS	Customer logged in
STEPS	1. Restart app 2. then invalidate refresh token and restart
EXPECTED	Valid session restores without login flash; invalid session returns to login and clears customer cache

MOB-04 | P1 | Home states

PRECONDITIONS	Logged in
STEPS	1. Observe loading, populated and no-booking/no-gym states 2. force API error and retry
EXPECTED	Each state is legible; retry succeeds; active bookings link to details

MOB-05 | P0 | Nearby discovery with permission

PRECONDITIONS	Location services available
STEPS	1. Tap Find gyms near me/Use my location 2. allow permission 3. change radius/amenities
EXPECTED	Coordinates are used once; nearby gyms ordered with distance; invalid radius disables filter

MOB-06 | P0 | Location denial fallback

PRECONDITIONS	Location permission not granted
STEPS	1. Deny/restrict location 2. search by city/state/name
EXPECTED	App explains fallback; city search remains functional; restricted state can open settings

MOB-07 | P1 | Discovery filters and pagination

PRECONDITIONS	More than 20 matching gyms
STEPS	1. Apply search/city/state/amenity filters 2. scroll 3. pull to refresh 4. switch nearby to city
EXPECTED	Correct rows load without duplicates; next page and refresh behave correctly

MOB-08 | P0 | Gym, branch and plan selection

PRECONDITIONS	Approved gym has active branches/plans
STEPS	1. Open gym 2. choose branch 3. inspect address/amenities/hours 4. choose active plan
EXPECTED	Plans are scoped to selected branch; inactive plans are not selectable; timezone is displayed

MOB-09 | P0 | Day-pass slot selection

PRECONDITIONS	Active day pass and slot inventory exist
STEPS	1. Choose day pass 2. select date 3. attempt full/blocked slot 4. select available slot
EXPECTED	Full/blocked slot disabled; available capacity and branch-local time are correct

MOB-10 | P1 | Membership plan without slot

PRECONDITIONS	Active monthly/quarterly/yearly plan exists
STEPS	1. Choose membership plan 2. continue to review
EXPECTED	Review opens without a slot and shows duration/visit limit and authoritative price

MOB-11 | P0 | Reserve with server authority

PRECONDITIONS	Valid plan/branch/slot
STEPS	1. Review 2. tap Reserve and continue once and repeatedly
EXPECTED	One idempotent booking is created; amount/currency come from server snapshot; status is PAYMENT_PENDING

MOB-12 | P0 | Concurrent last-slot protection

PRECONDITIONS	One place remains; two customers ready
STEPS	Submit both reservations at nearly the same time
EXPECTED	At most one capacity hold succeeds; availability never becomes negative

MOB-13 | P0 | Development payment journey

PRECONDITIONS	PAYMENT_PROVIDER=development; pending booking
STEPS	1. Continue to payment 2. note message 3. Admin simulates capture 4. refresh booking
EXPECTED	App never claims a charge; after authoritative capture, payment is SUCCESS and booking CONFIRMED

MOB-14 | P0 | Razorpay test checkout

PRECONDITIONS	Razorpay test credentials and native development build available
STEPS	1. Create order 2. complete test checkout 3. return to app
EXPECTED	Backend validates order, payment, signature, amount, currency and booking before confirmation; secrets never appear in client logs/UI

MOB-15 | P0 | Invalid payment proof

PRECONDITIONS	Pending booking
STEPS	1. Supply invalid signature/wrong order through test harness 2. refresh
EXPECTED	Verification fails with stable safe error; booking remains unconfirmed; no earning/ledger is created

MOB-16 | P0 | Reservation cancellation/expiry

PRECONDITIONS	Pending booking exists
STEPS	1. Cancel one booking 2. allow another to reach expiry 3. refresh slots
EXPECTED	Status becomes CANCELLED/EXPIRED; capacity is released exactly once; cancellation does not promise a refund

MOB-17 | P1 | Booking list and detail

PRECONDITIONS	Customer has mixed booking states
STEPS	1. Open Bookings 2. paginate/refresh 3. open pending, confirmed and refunded records
EXPECTED	Only own bookings appear; grouping/status, amount, slot timezone, payment and refund status are correct

MOB-18 | P1 | Profile update

PRECONDITIONS	Customer logged in
STEPS	1. Update name/email 2. try invalid or duplicate email 3. restart
EXPECTED	Valid profile persists; validation/API error shown safely; new display name appears after restore

MOB-19 | P0 | Customer booking IDOR

PRECONDITIONS	Two customers have bookings
STEPS	As Customer A, request Customer B booking/payment IDs
EXPECTED	Access is denied/not found and no B details leak

MOB-20 | P1 | Offline and recovery

PRECONDITIONS	Customer logged in
STEPS	1. Disable network during list load and mutation 2. reconnect 3. retry
EXPECTED	Offline/error state is visible; mutations are not silently queued or duplicated; manual retry recovers

10. Cross-surface integrity cases

INT-01 | P0 | Booking confirmation boundary

PRECONDITIONS	Pending booking/payment
STEPS	Attempt client-side success flag or direct booking status update
EXPECTED	Request is ignored/rejected; only verified payment path can confirm booking

INT-02 | P0 | Payment order idempotency

PRECONDITIONS	Unexpired pending booking
STEPS	Submit order creation concurrently with same/different client idempotency attempts
EXPECTED	One active provider order/payment is reused; no duplicate charge records

INT-03 | P0 | Duplicate/out-of-order webhook

PRECONDITIONS	Valid signed provider event available
STEPS	1. Deliver success twice 2. deliver delayed failure after success
EXPECTED	Event uniqueness prevents duplicate booking/earning/ledger effects; terminal success is not regressed

INT-04 | P0 | Invalid webhook signature

PRECONDITIONS	Raw provider payload available
STEPS	Change payload/signature and send
EXPECTED	Request rejected; event cannot change payment, booking, earning, or ledger

INT-05 | P0 | Atomic payment success

PRECONDITIONS	Fault injection in isolated test DB
STEPS	Fail the success transaction at each internal write boundary
EXPECTED	No persistent state has payment SUCCESS with missing booking confirmation/earning/ledger

INT-06 | P0 | Refund financial consistency

PRECONDITIONS	Successful earning exists
STEPS	1. Apply partial then full refunds 2. repeat provider callback
EXPECTED	Refunded total never exceeds captured amount; earning/commission/net and ledger reconcile without duplicates

INT-07 | P0 | Settlement concurrency

PRECONDITIONS	Eligible earnings exist
STEPS	Run two generation workers for same gym/period
EXPECTED	An earning belongs to at most one settlement item; totals remain deterministic

INT-08 | P1 | Monetary rounding

PRECONDITIONS	Amounts that do not divide evenly by commission BPS
STEPS	Complete payment/refund and inspect snapshots
EXPECTED	All calculations use integer minor units and deterministic rounding; historical snapshots do not change with later config

INT-09 | P0 | Sensitive data exposure

PRECONDITIONS	Complete representative web/mobile journeys
STEPS	Inspect UI, network responses and logs
EXPECTED	No JWT, refresh token, OTP hash, provider secret, provider payment secret, or internal stack is exposed

INT-10 | P1 | Accessibility and responsive use

PRECONDITIONS	Current Chrome/Safari and Android/iOS device sizes
STEPS	1. Use keyboard/screen reader on web 2. large text/touch navigation on mobile
EXPECTED	Labels, focus order, errors, confirmations, touch targets and core actions remain usable without clipped content

11. UAT execution record

Build/commit	
Environment	
Tester	
Browser/device/OS	
Test case ID	
Result	
Evidence	
Defect ID and notes	

12. Acceptance exit criteria

1	Every P0 case passes on the relevant surface or has an explicitly accepted release waiver.
2	No open authorization, payment-verification, financial-integrity, capacity or sensitive-data defect remains.
3	Partner A cannot access Partner B resources, and Customer A cannot access Customer B booking or payment details.
4	Admin and Partner web pass on supported desktop browsers; mobile passes on supported Android and a native iOS build before store readiness is claimed.
5	Razorpay success is claimed only after test credentials, signed callbacks and native checkout were actually validated.
6	Lint, typecheck, tests and builds pass; environment-dependent runtime checks are recorded separately.

END OF GUIDE